

Javascript: sintaxi, funcions, arrays, objectes

Anton Dalmau Mines

Índex

Introducció	5
Resultats d'aprenentatge	7
1 Sintaxi del llenguatge	9
1.1 Arquitectura de les aplicacions web. Javascript	9
1.1.1 Introducció	9
1.1.2 Naixement de JavaScript	13
1.2 Instal·lació de Node.js i npm	14
1.2.1 Instal·lació de Node.js	15
1.3 Sintaxi del llenguatge: variables, operadors, condicionals, bucles	16
1.3.1 Variables i tipus	16
1.3.2 Capturar dades	18
1.3.3 Operadors	19
1.3.4 Classes Core i Mòduls de JavaScript	22
1.3.5 Condicionals	26
1.3.6 Bucles (Iteracions)	27
2 Programació amb funcions	31
2.1 Funcions predefinides del llenguatge	31
2.2 Funcions definides per l'usuari	33
2.2.1 Declaració de funció	33
2.2.2 Funcions anònimes i assignació a variables	34
2.2.3 Funcions fletxa (Arrow Functions)	34
2.2.4 Paràmetres	36
2.2.5 Àmbit (Scope) de les variables	36
2.3 Conceptes avançats de funcions	37
2.3.1 Closures (funcions tancades)	37
2.3.2 Funcions com a objectes	38
2.3.3 Funcions com a fàbriques (closures + mètodes)	40
2.3.4 El context "this"	41
3 Programació amb arrays i objectes	43
3.1 Arrays. Objectes definits pel programador	43
3.1.1 Arrays com estructures de dades	43
3.1.2 Objectes com estructures de dades	45
3.1.3 Estructures de dades complexes	48
3.2 Programació orientada a objectes	50
3.2.1 Introducció a les classes	50
3.2.2 El paper de 'this' dins de les classes	51
3.2.3 Encapsulació i modificació de valors	51
3.2.4 Abstracció	53
3.2.5 Herència entre classes	54
3.2.6 Polimorfisme	55

3.2.7	Una menció als prototips	55
3.3	Exemples pràctics	56
3.3.1	Exemple 1: Estudiants i qualificacions	56
3.3.2	Exemple 2: Jerarquia d'animals	58

Introducció

Aquesta unitat introdueix els fonaments del llenguatge JavaScript, que constitueix la base de la programació al costat client en aplicacions web. S'hi treballa la sintaxi bàsica del llenguatge, la declaració de variables, els tipus de dades i les estructures de control, que permeten definir el flux d'execució d'un programa. Un element clau de la unitat és l'ús de funcions, tant com a mecanisme de reutilització de codi com a eina fonamental per estructurar programes més complexos. També s'introdueixen els *arrays*, que permeten gestionar col·leccions de dades, i els objectes, essencials per modelar informació de manera estructurada. Al llarg de la unitat es posa èmfasi en la comprensió del funcionament intern del llenguatge i en l'escriptura de codi clar i llegible. Aquest coneixement és imprescindible per afrontar la resta d'unitats del curs, ja que proporciona les eines bàsiques necessàries per treballar amb esdeveniments, el DOM i la programació asíncrona.

Resultats d'aprenentatge

En finalitzar aquesta unitat, l'alumne/a:

1. Selecciona les arquitectures i tecnologies de programació sobre clients web, identificant i analitzant les capacitats i les característiques de cadascuna.

- Caracteritza i diferencia els models d'execució de codi al servidor i al client web.
- Identifica les capacitats i els mecanismes d'execució de codi dels navegadors web.
- Identifica i caracteritza els principals llenguatges relacionats amb la programació de clients web.
- Identifica i caracteritza els principals llenguatges relacionats amb la programació de clients web.
- Reconeix les particularitats de la programació de guions i els seus avantatges i desavantatges sobre la programació tradicional.
- Verifica els mecanismes d'integració dels llenguatges de marques amb els llenguatges de programació de clients web.
- Reconeix i avalua les eines de programació i prova sobre clients web.

2. Escriu sentències simples, aplicant la sintaxi del llenguatge i verificant la seva execució sobre navegadors web.

- Selecciona un llenguatge de programació de clients web a funció de les possibilitats.
- Utilitza els diferents tipus de variables i operadors disponibles al llenguatge.
- Identifica els àmbits d'utilització de les variables.
- Reconeix i comprova les peculiaritats del llenguatge respecte a les conversions entre diferents tipus de dades.
- Utilitza mecanismes de decisió en la creació de blocs de sentències.
- Utilitza bucles i verifica el funcionament.
- Afegeix comentaris al codi.
- Utilitza eines i entorns per facilitar la programació, la prova i la documentació del codi.

3. Escriu codi, identificant i aplicant les funcionalitats aportades pels objectes predefinits del llenguatge.

- Identifica els objectes predefinits del llenguatge.
- Analitza els objectes referents a les finestres del navegador i els documents web que hi contenen.
- Escriu sentències que utilitzin els objectes predefinits del llenguatge per canviar l'aspecte del navegador i el document que hi conté.
- Genera textos i etiquetes com a resultat de l'execució de codi al navegador.
- Escriu sentències que utilitzin els objectes predefinits del llenguatge per interactuar amb l'usuari.
- Utilitza les característiques pròpies del llenguatge en documents compostos per diverses finestres.
- Utilitza mecanismes del navegador web per emmagatzemar informació i recuperar-ne el contingut.
- Depura i documenta el codi.

4. Programa codi per a clients web analitzant i utilitzant estructures definides per l'usuari.

- Classifica i utilitza les funcions predefinides del llenguatge.
- Crea i utilitza funcions definides per l'usuari.
- Reconeix les característiques del llenguatge relatives a la creació i l'ús de matrius (*arrays*).
- Crea i utilitza matrius (*arrays*).
- Utilitza operacions agregades per al maneig d'informació emmagatzemada a col·leccions.
- Reconeix les característiques d'orientació a objectes del llenguatge.
- Crea codi per definir l'estructura d'objectes.
- Crea mètodes i propietats.
- Crea codi que faci ús d'objectes definits per l'usuari.
- Utilitza patrons de disseny de programari.
- Depura i documenta el codi.

1. Sintaxi del llenguatge

En aquest capítol introduïrem la sintaxi bàsica del llenguatge **JavaScript**, el qual és un dels pilars fonamentals del desenvolupament web modern. Aprendrem com s'integra dins l'arquitectura de les aplicacions web, com instal·lar i configurar l'entorn de treball amb **Node.js** i **npm**, i com escriure les primeres instruccions i estructures bàsiques del llenguatge.

L'objectiu és entendre la forma i l'estructura del codi JavaScript, així com les convencions de nomenclatura i els elements essencials que utilitzarem en el curs.

1.1 Arquitectura de les aplicacions web. Javascript

En aquest primer bloc ens endinsarem en la sintaxi del llenguatge JavaScript, un dels pilars fonamentals del desenvolupament web modern. Abans, però, és important entendre en quin context apareix i per què és tan rellevant. La web actual ja no és un conjunt de pàgines estàtiques, sinó un ecosistema viu i dinàmic on les aplicacions interactuen constantment amb l'usuari i amb altres serveis. Per això, més enllà del simple maquetat amb **HTML** i l'estilització amb **CSS**, necessitem un llenguatge capaç d'aportar lògica, dinamisme i interactivitat: aquest llenguatge és **JavaScript**.

L'objectiu d'aquesta lliçó és proporcionar-nos una base sòlida sobre la sintaxi i el funcionament de JavaScript, començant per la seva ubicació dins l'arquitectura de les aplicacions web i seguint després amb els conceptes essencials que utilitzarem al llarg de tot el curs.

1.1.1 Introducció

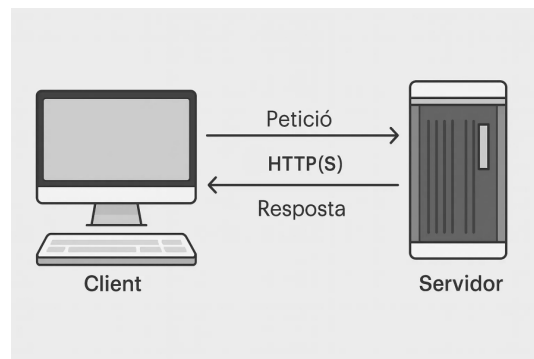
Avui en dia, tenim una gran quantitat de dispositius connectats al núvol. El portàtil, el telèfon mòbil, la tauleta, la televisió..., i amb l'*Internet of Things* (IoT), cada vegada aquests dispositius aniran en augment. Una de les coses habituals que fem amb algun d'aquests dispositius és navegar per la web tot visitant pàgines web.

D'altres dispositius i aplicacions, tot i que no serveixen per navegar per la web, sí que fan consultes a serveis web. Imaginem per exemple un aplicatiu web per controlar una casa domòtica, on podem posar la calefacció a distància. O una aplicació per al mòbil de calendari, que se sincronitza contínuament i ens avisa de quan tenim una reunió. Per tant, hem de prendre consciència del fet que el que ens permet la tecnologia web, avui dia, és molt més que visitar pàgines web.

Si una cosa tenen en comú tots aquests aplicatius web és que es basen en l'**arquitectura client-servidor**. Quan parlem de client web ens ve el cap el navegador web (Firefox, Chrome, Internet Explorer, etc.). Ara bé, una aplicació que ens permet consultar l'horari de la parada del bus també és un client web, tot i que la presentació no té format de pàgina web.

A l'altra banda tenim els servidors web, que serveixen el contingut web que tenen allotjat. Exemples de servidor web són l'Apache Web Server o l'Internet Information Server, però també el servidor d'aplicacions Tomcat (vegeu la figura 1.1).

FIGURA 1.1. Arquitectura client-servidor en la comunicació HTTP



Suposem que estem amb el nostre portàtil, i consultem la web de l'IOC des del navegador Chrome. Fixem-nos amb el camí que fan els paquets IP. Des del Chrome, en la barra de navegació, posem una URL que identifica unívocament el contingut que volem baixar (per exemple, www.ioc.xtec.cat). Aquesta informació es troseja en forma de paquets IP que viatgen per Internet. Els servidors DNS localitzen la direcció IP del servidor on està allotjat el contingut. A més, aquests paquets han de passar per *routers*, *firewalls* i diferents capes de seguretat.

Els paquets IP finalment s'ajunten en el destí, on tenim un servidor web Apache, per exemple, que allotja l'script *index.php*. Aquest script s'executa (es processa la part de PHP i s'ajunta amb la part d'HTML), i finalment el resultat se serveix al destinatari fent el camí invers.

Quan la pàgina web arriba al destí, en el navegador web podem visualitzar la informació demanada. El navegador web, el nostre client, és el responsable de maquetar correctament la pàgina web a partir del contingut HTML, els fulls d'estil CSS, i també el codi JavaScript que conté. Podem veure el codi font que hem rebut del servidor anant a l'inspector d'elements del navegador (fixem-nos que mai es veurà en el codi font el codi PHP; per contra, sí que podem veure el codi JavaScript).

L'entorn de client que estem estudiant és precisament la part del navegador web, en contraposició amb la part de servidor. Així com el codi PHP s'executa en el servidor, el codi JavaScript que contenen les webs s'executa en el client.

Als inicis d'Internet (anys 90 del s. XX), les pàgines web estaven formades únicament per documents **HTML estàtics**, que només permetien mostrar **text pla**

i **imatges**. No hi havia cap mena d'interactivitat amb l'usuari ni possibilitat de resposta dinàmica per part del navegador.

Amb el pas del temps, i especialment amb l'aparició de la **Web 2.0**, aquest panorama ha canviat radicalment. Les pàgines actuals són **dinàmiques, interactives** i amb una gran càrrega **visual**. Aquest canvi ha estat possible gràcies a la incorporació de **JavaScript**, un llenguatge de programació que s'executa directament en el navegador de l'usuari (cantó del client).

JavaScript ha esdevingut una **tecnologia clau** en el desenvolupament web modern per molts motius:

- **Llenguatge natiu dels navegadors.** JavaScript és **l'únic llenguatge** que tots els navegadors web entenen i poden executar de manera nativa. Per utilitzar altres llenguatges en el costat client (com TypeScript, Dart o Elm), és necessari utilitzar un **transpilador** que els converteixi prèviament a JavaScript.
- **Efectes visuals i dinamisme.** Permet aplicar una gran varietat d'**efectes visuals** que milloren l'experiència de l'usuari, com:
 - Aparició i desaparició de capes amb efectes de dissolució
 - Desplegables o continguts que es col·lapsen
 - Finestrals emergents (pop-ups)
 - Transicions d'imatges i galeries interactives
 - *Scrolls* animats, *sliders* i menús dinàmics
- **Interactivitat amb l'usuari.** JavaScript permet respondre a accions de l'usuari en temps real: clics, tecles, moviments del ratolí, etc. Això fa possible construir interfícies riques i intuïtives.
- **Validació de formularis en el client.** Amb JavaScript es pot validar la informació d'un formulari **abans d'enviar-la al servidor** (això millora la usabilitat, redueix el temps de resposta i evita càrregues innecessàries al servidor), per exemple:
 - Verificar el format d'un DNI, número de telèfon o correu electrònic
 - Comprovar que un camp no estigui buit
 - Validar dates o codis postals
- **Accés a informació del dispositiu i del navegador.** JavaScript pot accedir a dades del navegador, com:
 - Sistema operatiu
 - Idioma del navegador
 - Resolució de pantalla
 - Estat de connexió a Internet
- **Desenvolupament d'extensions i eines.** JavaScript és la base per crear **extensions** per navegadors com **Chrome**, **Firefox** o **Edge**, que permeten ampliar la funcionalitat del navegador amb eines personalitzades.

Transpilador

Un transpilador és una eina que converteix codi font escrit en un llenguatge de programació a un altre llenguatge de programació similar, mantenint habitualment el mateix nivell d'abstracció.

- **Aplicacions web avançades:** Gràcies a JavaScript i als seus *frameworks* (React, Angular, Vue,...), és possible crear **aplicacions web complexes** amb funcionalitats pròpies d'un programari d'escriptori. Alguns exemples destacats són:
 - **Google Docs** (processador de textos i fulls de càlcul en línia)
 - **Figma** (disseny d'interfícies col·laboratiu)
 - **Canva** (editor gràfic al núvol)
 - **Jocs:** per exemple, bit.ly/3hoeO7T

Tot i que **JavaScript** és un llenguatge de programació que pot executar-se fora del context web (com veurem més endavant amb Node.js), el seu gran èxit s'ha produït gràcies a la web. És una de les tecnologies fonamentals per desenvolupar **pàgines web dinàmiques i interactives**.

Per començar a treballar amb JavaScript dins d'una pàgina HTML, hem de veure com conviuen la **sintaxi HTML** i la **sintaxi JavaScript**. JavaScript s'integra en les pàgines web mitjançant l'etiqueta `<script>`.

L'atribut `type="text/javascript"` que sovint apareix en exemples antics ja no és necessari en HTML5, ja que JavaScript és el llenguatge de script per defecte.

Exemple bàsic de pàgina HTML i interacció amb l'usuari

El codi que veurem a continuació el podem escriure amb un editor de text o amb un IDE com per exemple Visual Studio Code. Després el podem desar dins d'una carpeta determinada.

```
1  <!DOCTYPE html>
2  <html lang="ca">
3    <head>
4      <meta charset="UTF-8">
5      <title>El meu primer exemple</title>
6      <script>
7        function actualitzaParagraf() {
8          document.getElementById("paragraf").innerText = "El contingut del
          paràgraf ha canviat!";
9        }
10     </script>
11   </head>
12   <body>
13     <h1>El meu primer exemple</h1>
14     <p id="paragraf">Aquest és el text original del paràgraf.</p>
15     <button onclick="actualitzaParagraf()">Clica'm</button>
16   </body>
17 </html>
```

Quan obrim aquesta pàgina en un navegador (per exemple Chrome o Firefox), veurem un paràgraf i un botó. Quan fem clic al botó, el contingut del paràgraf canviarà. Aquest és el comportament bàsic que permet JavaScript: **interacció amb l'usuari**.

Explicació del funcionament:

- El botó HTML incorpora l'atribut `onclick`, que **dispara un esdeveniment** quan l'usuari hi fa clic.
- Aquest esdeveniment executa la funció `actualitzaParagraf()`, definida dins de l'etiqueta `<script>`.
- La funció localitza l'element amb identificador `paragraf` (mitjançant `document.getElementById`) i modifica el seu contingut textual.

Ubicació de l'etiqueta `<script>`:

Tot i que en l'exemple anterior hem situat el codi JavaScript dins del `<head>`, també és **perfectament vàlid i sovint recomanable** col·locar-lo **al final del** `<body>`. Això assegura

que l'HTML estigui carregat abans que s'executi el codi JavaScript, evitant errors si l'script accedeix a elements que encara no existeixen en el DOM.

```
1 <!DOCTYPE html>
2 <html lang="ca">
3   <head>
4     <meta charset="UTF-8">
5     <title>El meu primer exemple</title>
6   </head>
7   <body>
8     <h1>El meu primer exemple</h1>
9     <p id="paragraf">Aquest és el text original del paràgraf.</p>
10    <button onclick="actualitzaParagraf()">Clica'm</button>
11
12    <script>
13      function actualitzaParagraf() {
14        document.getElementById("paragraf").innerText = "El contingut del
15        paràgraf ha canviat!";
16      }
17    </script>
18  </body>
19 </html>
```

Dins l'etiqueta `<script>` s'hi troba el nostre codi. Sovint, sobretot si el codi es llarg, aquest s'allotja en un fitxer extern amb extensió `.js` i s'enllaça de la següent manera:

```
1 <script src="elMeuScript.js"></script>
```

Com provar el codi:

- 1) Obrirem un **editor de text pla** (com Notepad, Gedit, VS Code, Sublime Text3, etc.).
- 2) Copiem-hi l'exemple HTML anterior i el desarem com `exemple.html`.
- 3) Obrim aquest fitxer amb el navegador. Podem fer-ho arrossegant-lo al navegador, fent doble click sobre l'arxiu, amb el botó dret del ratolí i l'opció *obrir amb*, o escrivint la ruta al navegador:

```
1 file://ruta_al_fitxer/exemple.html
```

- 4) Veurem com, en clicar el botó, el contingut del paràgraf es modifica: la nostra **primera interacció JavaScript** ha tingut lloc!

Aquesta pràctica introdueix conceptes essencials com l'ús d'**esdeveniments**, **funcions**, i la **manipulació del DOM (Document Object Model)**, que són fonaments de qualsevol aplicació web moderna.

1.1.2 Naixement de JavaScript

JavaScript va ser creat per **Netscape** i, el 1997, es va proposar com a estàndard a l'**ECMA** (European Computer Manufacturers Association). Així va néixer **ECMAScript**, el llenguatge base de JavaScript, avui dia reconegut com a estàndard **ISO/IEC**.

Els problemes de compatibilitat entre navegadors (Internet Explorer, Netscape Navigator, Opera) van impulsar la necessitat d'estandardització. El **W3C (World Wide Web Consortium)** va establir el **DOM (Document Object Model)** per garantir un comportament coherent dels elements HTML/XML a tots els navegadors.

Avui en dia, **JavaScript és un dels llenguatges més importants i utilitzats del desenvolupament web**, i va molt més enllà de la seva funció original

com a eina per fer pàgines web interactives. Amb l'aparició de *frameworks* i biblioteques com **React**, **Angular**, **Vue** o **Svelte**, JavaScript ha esdevingut l'eix central per a la creació d'aplicacions web modernes, conegudes com a *Single Page Applications* (SPA). Aquestes aplicacions permeten experiències d'usuari fluides, amb actualitzacions dinàmiques del contingut sense necessitat de recarregar la pàgina.

A més, gràcies a **Node.js**, JavaScript també s'utilitza en el costat del servidor, fet que permet desenvolupar aplicacions completes amb un únic llenguatge, tant al frontend com al backend. Això ha propiciat l'aparició de noves arquitectures, com la **MERN stack** (MongoDB, Express, React, Node.js), molt popular en projectes web i *startups*. Amb eines com **npm** (*Node Package Manager*), la comunitat de desenvolupadors disposa d'un ecosistema immens de paquets i recursos que acceleren la creació d'aplicacions escalables, modernes i mantenibles.

1.2 Instal·lació de Node.js i npm

Tot i que **JavaScript** és conegut principalment pel seu ús en desenvolupament web, cal remarcar que **no és exclusiu de la web**. Es tracta d'un llenguatge de programació generalista, amb entorns d'execució independents del navegador que permeten desenvolupar tota mena d'aplicacions.

Quan s'executa JavaScript dins d'un navegador (com Firefox, Chrome o Safari), el codi és processat per un **motor de JavaScript** integrat en el mateix navegador (vegeu la taula 3.1).

TAULA 1.1. Motors de JavaScript destacats

Motor	Navegador / Entorn	Llenguatge d'implementació
SpiderMonkey	Firefox	C/C++
V8	Chrome, Edge, Node.js	C++
JavaScriptCore	Safari	C++
Chakra (antic)	Internet Explorer	C++

Una de les tecnologies més revolucionàries que han consolidat JavaScript fora del navegador és **Node.js**

Node.js és un entorn d'execució que utilitza el motor **V8** de Google (el mateix que fa servir Chrome) per executar codi JavaScript en el servidor. Va ser creat per Ryan Dahl el 2009, i permet utilitzar JavaScript per crear:

- Servidors web
- Aplicacions de xarxa (com xats en temps real)
- Scripts de línia d'ordres
- Automatitzacions i eines de desenvolupament

- APIs RESTful i serveis backend

Les característiques clau de Node.js són:

- Basat en **esdeveniments** i **programació asíncrona**
- Lleuger, ràpid i escalable
- Disposa del **gestor de paquets npm**, amb milers de biblioteques

1.2.1 Instal·lació de Node.js

A continuació, es detalla com instal·lar **Node.js** (i el seu gestor de paquets **npm**) sota la plataforma Windows (de manera anàloga es fa amb altres sistemes operatius):

1. Anem a la web oficial, nodejs.org.
2. Triem la **versió LTS (Long Term Support)** per garantir estabilitat.
3. Descarreguem i executem l'instal·lador.
4. Durant la instal·lació:
 - Acceptem els termes d'ús.
 - Marquem l'opció que afegeix Node.js al `PATH` (sol estar marcada per defecte).
 - Continuem endavant fins que finalitzi.
5. Un cop acabat, obrim una nova terminal i comprovem les versions que hem instal·lat:

```
1 node -v
2 npm -v
```

Així verifiquem que tant node com npm (el gestor de paquets) funcionen correctament.

Exemple bàsic amb Node.js

A continuació, es mostra un exemple simple d'un **servidor HTTP** escrit amb Node.js:

```
1 const http = require('http');
2
3 const server = http.createServer((req, res) => {
4   res.statusCode = 200;
5   res.setHeader('Content-Type', 'text/plain; charset=utf-8');
6   res.end('Hola món des de Node.js!');
7 });
8
9 server.listen(3000, () => {
10   console.log('Servidor en funcionament a <http://localhost:3000>');
11 });
```

Per executar aquest servidor:

- 1) Guardem el codi en un fitxer anomenat **server.js**.
- 2) Obrim una terminal en el directori on hem desat l'script `server.js` i executem: `node server.js`.

Visitem <http://localhost:3000> al navegador per veure la resposta.

1.3 Sintaxi del llenguatge: variables, operadors, condicionals, bucles

Un cop entès el paper de JavaScript dins de l'arquitectura web i coneguts els seus orígens, és el moment de centrar-nos en la seva **sintaxi bàsica**. La sintaxi és el conjunt de regles que defineixen com s'ha d'escriure i estructurar el codi perquè el navegador (o qualsevol entorn d'execució) el pugui interpretar correctament. Aprendre aquests fonaments és imprescindible per començar a programar, ja que constitueixen les peces bàsiques amb què construirem instruccions més complexes i aplicacions completes.

1.3.1 Variables i tipus

JavaScript és un llenguatge **feblement tipat**, és a dir, no indiquem de quin tipus és cada variable quan la declarem. Totes les variables poden contenir qualsevol tipus de dada i poden ser reassignades. Això té els seus avantatges i inconvenients.

Les variables són espais de memòria on emmagatzemem dades de manera temporal, i hi podem accedir en qualsevol moment durant l'execució del nostre programa. Hi ha diversos tipus i classes, que veurem tot seguit.

Per definir una variable en JavaScript, utilitzem la paraula reservada *let* i li donem un nom:

```
1 let elMeuValor;
```

També podem assignar-li un valor directament:

```
1 let dada = 5;
```

Com mostrar dades a l'usuari?

Per mostrar dades a l'usuari, tenim diferents mecanismes. Els més bàsics són mostrar-los per consola (`console.log(dada)`) o si ens trobem en un document web els podem mostrar per pantalla (`document.write(dada)`) o fins i tot fer-los sortir en una finestra emergent del navegador (`alert(dada)`).

O fer-ho en una línia posterior:

```
1 let dada;  
2 dada = 5;
```

Al llarg de les lliçons, sovint veurem declarar variables amb la partícula *const*. Això serveix per declarar una variable amb un valor constant i no podrà ser canviat excepte que la variable sigui de tipus objecte o array. Per exemple:


```
1 const PI = 3.1416;
```

Finalment, també val la pena conèixer que antigament es declaraven les variables amb la partícula `var`. Malgrat que encara funciona es desaconsella el seu ús.

Els noms de les variables haurien de ser el més **descriptius** possible per entendre millor què contenen. Han de començar sempre per una lletra, `$` o `_`, però **mai per números** ni altres caràcters especials. JavaScript distingeix entre majúscules i minúscules, per tant, `meuValor` i `MeuValor` serien dues variables diferents.

JavaScript disposa de **set tipus primitius** per emmagatzemar dades:

- **number**: s'utilitza per a **valors numèrics**, tant enters com decimals. Exemples:

```
1 let numEnter = 5;  
2 let numDecimal = 3.14;  
3 console.log(typeof numDecimal); // "number"
```

- **boolean**: només pot contenir `true` o `false`, i s'utilitza per representar estats:

```
1 let on = true;  
2 let off = false;
```

- **string**: representa cadenes de caràcters, entre cometes simples o dobles:

```
1 let missatge = "Hola món";  
2 let altreMissatge = 'Bon dia';
```

- **undefined**: És el valor que pren una variable quan **no se li ha assignat cap valor**, o s'ha assignat explícitament `undefined`:

```
1 let noDefinida;  
2 let tambéIndefinida = undefined;
```

- **null**: valor assignat que representa **absència intencionada de valor**:

```
1 let resposta = null;  
2 console.log(resposta); // null
```

- **bigint**: per treballar amb enters molt grans:

```
1 let granNumero = 1234567890123456789012345678901234567890n;  
2 console.log(granNumero); // →  
   1234567890123456789012345678901234567890n  
3 console.log(typeof granNumero); // "bigint"
```

- **symbol**: identificadors únics, sovint per propietats d'objectes:

```
1 const id = Symbol("usuariID");  
2 console.log(id); // Symbol(usuariID)  
3 console.log(typeof id); // "symbol"
```

Altres tipus

A part dels tipus primitius, JavaScript també utilitza **objectes** per representar estructures de dades més complexes. Tot en JavaScript és un objecte (incloses les funcions). Alguns objectes comuns són:

- Object
- Array
- Date

Tots ells hereten de la classe Object.

1.3.2 Capturar dades

Els valors desats en les variables estudiades a la secció anterior poden haver estat introduïts pels desenvolupadors o bé provenir directament de l'usuari. Capturar informació de l'usuari és fonamental per fer programes interactius i adaptatius. Tot i que la forma més habitual en aplicacions web són els formularis (que tractarem més endavant), hi ha altres mètodes senzills i molt útils per obtenir dades, tant en entorns de navegador com en NodeJS.

Des del navegador

En JavaScript que s'executa al navegador podem interactuar amb l'usuari de manera immediata a través de finestres emergents:

- **prompt('Pregunta')**. Permet mostrar un quadre de diàleg amb una pregunta i un camp de text per introduir dades. Retorna sempre una **cadena** de text, encara que l'usuari escrigui un número.

```
1 let nom = prompt('Com et dius?');  
2 console.log('Hola, ' + nom + '!');
```

- **confirm('Estàs segur?')**. Mostra un quadre de diàleg amb els botons **OK** i **Cancel**, retornant un valor **boolean** (true si l'usuari prem OK, false si prem Cancel).

```
1 let confirmar = confirm('Vols continuar amb l'acció?');  
2 if (confirmar) {  
3     console.log('Acció confirmada.');
```

- **alert('Missatge')**. Tot i que no captura dades, és útil per mostrar informació immediata a l'usuari. Sovint es combina amb prompt o confirm.

```
1 alert('Benvingut a la nostra pàgina!');
```

Des de la consola amb NodeJS

En entorns fora del navegador, com NodeJS, podem capturar dades de l'usuari a través de la consola. La manera més senzilla és mitjançant el mòdul `readline`:

```
1 const readline = require('readline');
2
3 const rl = readline.createInterface({
4   input: process.stdin,
5   output: process.stdout
6 });
7
8 rl.question('Com et dius? ', (nom) => {
9   console.log('Hola, ${nom}!');
10  rl.close();
11 });
```

- `readline.createInterface` crea un canal per llegir i escriure dades a la consola.
- `rl.question` mostra la pregunta i espera que l'usuari introdueixi una resposta.
- La resposta es passa a una funció de *callback*, on podem processar-la i emmagatzemar-la en una variable.

A més, podem capturar dades de forma més avançada, com per exemple llegir múltiples preguntes en seqüència o processar entrades contínues, però això ja forma part d'usos més avançats que estudiarem més endavant.

1.3.3 Operadors

JavaScript disposa d'**operadors** per a treballar amb tipus primitius i objectes. Ens permeten formar expressions. Els més habituals són els **aritmètics**, però també n'hi ha d'altres.

Operadors aritmètics

Serveixen per fer càlculs matemàtics:

```
1 5 + 2    // Suma → 7
2 5 - 2    // Resta → 3
3 (3 + 2) - 5 // Parèntesis → 0
4 3 / 3    // Divisió → 1
5 6 * 3    // Multiplicació → 18
```

L'operador + també serveix per **concatenar** cadenes de text:

```
1 "Hola " + "món" + "!" // Resultat: "Hola món!"
```

També tenim operadors d'**increment** i **decrement** (pre i post):

```
1 let x = 1;  
2 let y = ++x; // x = 2, y = 2  
3 let z = y++ + x; // z = 4, y = 3, x = 1
```

Operador typeof

Aquest operador especial ens permet saber **quin tipus de dada** conté una variable. És un operador *unari* (s'aplica a un sol operand):

```
1 typeof 5 // "number"  
2 typeof false // "boolean"  
3 typeof "Joan" // "string"  
4 typeof undefined // "undefined"
```

Operadors booleans

Només tenen dos valors possibles: `true` i `false`. Però poden modificar-se amb diversos operadors:

Negació

Canvia el valor a l'oposat:

```
1 !true // false  
2 !false // true  
3 !!true // true (doble negació)
```

Igualtat i identitat:

- `==` → Igualtat feble (comprova si són iguals, permet conversió de tipus)
- `===` → Identitat estricta (comprova valor i tipus)

```
1 true === true // true  
2 true === false // false  
3 true !== false // true  
4 true !== true // false
```

Es recomana fer servir `===` i `!==` en comptes de `==` i `!=` per evitar errors inesperats.

Error comú

Un error comú és pensar que aquesta expressió és falsa:

```
1 25 == "25"
```

En realitat són iguals, però no idèntics. L'expressió que retornaria `false` seria:

```
1 25 === "25"
```

Comparació

Podem comparar valors amb:

```
1 5 > 3      // true
2 5 < 3      // false
3 3 >= 3     // true
4 2 <= 1     // false
5 "a" < "b" // true
```

Valors que es consideren 'false' en JavaScript:

Valors falsy en JavaScript

En JavaScript, quan fem servir una expressió dins d'una condició (if, while, etc.), qualsevol valor es converteix implícitament a `true` o `false`. Els valors que s'avaluen com a `false` s'anomenen **falsy values**.

Els únics valors que es consideren **falsy** són:

- `false` (el valor booleà fals)
- `0` i `-0` (el zero numèric)
- `""` (cadena buida)
- `null`
- `undefined`
- `NaN` (Not a Number)
- `0n` (BigInt zero)

Tots els altres valors es consideren **truthy** (encara que siguin sorprenents, com per exemple `"0"`, `[]` o `{}.`

Operadors lògics

AND

Només retorna `true` si **tots dos valors** ho són:

```
1 true && true   // true
2 true && false  // false
```

Aquest operador (`&&`) s'utilitza molt per verificar condicions sense modificar els valors. Per exemple, és útil per comprovar si una propietat existeix.

La lògica que segueix és la següent:

- Si el **primer valor** és `false`, es retorna aquest valor.
- Si el **primer valor** és `true`, es retorna el segon valor.

```
1 0 && true // Retorna: 0 → perquè el 0 es considera un valor "
    fals"
2 1 && "Hola" // Retorna: "Hola" → perquè l'1 (o qualsevol valor
    diferent de 0) es considera "cert"
```

OR

Retorna el **primer valor cert**, o false si tots són falsos:

```
1 true || false // true
2 false || false // false
3 process.env.PORT || 5000 // Si process.env.PORT no està definit
    , s'utilitza 5000 com a valor per defecte
```

1.3.4 Classes Core i Mòduls de JavaScript

A més dels **tipus primitius** com number, string, boolean i undefined, JavaScript disposa de **classes bàsiques incorporades** que ens permeten treballar amb estructures de dades més complexes. Aquestes classes formen part del nucli del llenguatge (Core).

Les més habituals són:

- Object
- Number
- Date
- Math
- String
- Array

Totes elles **hereten de Object** i tenen els seus propis **mètodes i propietats**.

Object

És la classe més genèrica. Un objecte és un conjunt de **propietats (clau-valor)**, i pot contenir també **funcions** (anomenades mètodes).

```
1 let llibre = {
2   titol: "JavaScript per a tothom",
3   autor: "Ada Codaire",
4   pàgines: 240,
5   llegit: false
6 };
7
8 llibre.titol // "JavaScript per a tothom"
```

```
9 llibre["pàgines"] // 240
10 llibre.llegit = true; // assignació
```

Number

L'objecte Number representa **valors numèrics**, tant enters com decimals, i ofereix mètodes útils per manipular-los.

Creació de números:

```
1 let enter = 42;
2 let decimal = 3.1415;
3 let hexadecimal = 0xFF; // 255 en decimal
4 let exponencial = 5.4e2; // 540
```

També es pot crear amb el constructor:

```
1 let n = new Number(7); // No gaire recomanable, millor forma
  primitiva
```

Valors especials:

```
1 1 / 0 // Infinity
2 -1 / 0 // -Infinity
3 "a" / 2 // NaN (Not a Number)
```

Mètodes de l'objecte Number:

```
1 let n = 2.5674;
```

- `n.toFixed(2)` → "2.57" (arrodoneix amb 2 decimals)
- `n.toExponential(2)` → "2.57e+0" (forma científica)
- `(15).toString(2)` → "1111" (binari)
- `(255).toString(16)` → "ff" (hexadecimal)

Els valors numèrics en JavaScript tenen un rang aproximat de:

- `Number.MAX_VALUE` → valor màxim ($\sim 1.79 \times 10^{308}$)
- `Number.MIN_VALUE` → valor mínim ($\sim 5 \times 10^{-324}$)
- `Number.NaN`, `Number.POSITIVE_INFINITY`, `Number.NEGATIVE_INFINITY`

Date

L'objecte Date ens permet **treballar amb dates i hores**.

Crear una data:

```
1 let ara = new Date();           // Data i hora actual
2 let dataEspecifica = new Date(2025, 6, 17); // 17 de juliol de
   2025 (mesos compten des de 0!)
```

Obtenir components d'una data:

```
1 ara.getFullYear(); // 2025
2 ara.getMonth();    // 6 (juliol)
3 ara.getDate();     // 17
4 ara.getDay();      // 4 (dijous, sent 0 = diumenge)
5 ara.getHours();    // hora actual
6 ara.getMinutes();  // minuts actuals
```

Format com a cadena:

```
1 ara.toString();      // "Thu Jul 17 2025 10:32:00 GMT+0200"
2 ara.toLocaleDateString(); // "17/7/2025"
```

Math

És un **objecte estàtic** que proporciona funcions i constants matemàtiques. No cal crear una instància (`new Math()` no s'utilitza).

Constants útils:

```
1 Math.PI      // 3.141592...
2 Math.E       // 2.718281...
```

Funcions matemàtiques bàsiques:

```
1 Math.round(6.7); // 7 (arrodoniment)
2 Math.floor(6.7); // 6 (arrodoniment a la baixa)
3 Math.ceil(6.1);  // 7 (arrodoniment a l'alça)
4 Math.abs(-5);    // 5 (valor absolut)
5 Math.pow(2, 3);  // 8 (potència)
6 Math.sqrt(16);   // 4 (arrel quadrada)
```

Aleatorietat:

```
1 Math.random(); // nombre aleatori entre 0 i 1
2 Math.floor(Math.random() * 10); // enter aleatori entre 0 i 9
```

Trigonometria i logaritmes:

```
1 Math.sin(Math.PI / 2); // 1
2 Math.cos(0);           // 1
3 Math.tan(Math.PI / 4); // ~1
4 Math.log(10);          // logaritme natural
```

Array

Un Array (matriu) és una **col·lecció ordenada de valors**. Aquests valors poden ser nombres, cadenes, booleans, objectes, altres arrays, etc.


```
1 let buida = [];  
2 let numeros = [1, 2, 3];  
3 let textos = ["Hola", "adeu"];  
4 let mesclat = [1, "dos", true, [4, 5]];
```

També es pot crear amb el constructor:

```
1 let altra = new Array(1, 2, 3);
```

En lliçons posteriors es treballaran els arrays de forma més exhaustiva.

String

Una String és una seqüència de caràcters. Es defineix amb cometes simples, dobles o *backticks* (template strings).

Creació de strings:

```
1 let salutacio = "Hola món";  
2 let nom = 'Joan';  
3 let complet = `Hola, ${nom}!`; // "Hola, Joan!"
```

Accés als caràcters:

```
1 salutacio[0]; // "H"  
2 salutacio.charAt(1); // "o"  
3 salutacio.length; // 8
```

Mètodes útils:

```
1 // Transforma el text a majúscules  
2 "Hola".toUpperCase(); // "HOLA"  
3 // Transforma el text a minúscules  
4 "Hola".toLowerCase(); // "hola"  
5 // Cerca un text donat dins una cadena de text i retorna 'true'  
  si el troba  
6 "Hola món".includes("món"); // true  
7 // Indica la primera posició de l'string donat dins una cadena  
  de text  
8 "Hola món".indexOf("m"); // 5  
9 // Retorna el text comprès entre el rang donat  
10 "Hola món".substring(0, 4); // "Hola"  
11 // Retorna el text sense els espais en blanc  
12 " text ".trim(); // "text"
```

Convertir un string en array:

```
1 let paraules = "un dos tres".split(" "); // ["un", "dos", "tres"]
```

1.3.5 Condicionals

Les estructures condicionals ens permeten **executar diferents blocs de codi** en funció d'una condició determinada. D'aquesta manera podem **controlar el flux del programa** segons diferents casos o estats.

Assignació condicional (Operador ternari)

És una forma **compacta** de fer una assignació en funció d'una condició:

```
1 condició ? valor_si_cert : valor_si_fals
```

Exemples:

```
1 let numero = 10;  
2 let mesGranQueCinc = numero > 5 ? "sí": "no";  
3 console.log(mesGranQueCinc) // sí
```

Sentència if

S'utilitza per executar codi només si una condició és certa:

```
1 let nota = 8;  
2  
3 if (nota >= 5) {  
4   console.log("Aprovat");  
5 }
```

Sentència if / else

Permet executar un bloc si la condició és **certa**, i un altre bloc si és **falsa**:

```
1 let nota = 8;  
2  
3 if (nota >= 5) {  
4   console.log("Aprovat");  
5 } else {  
6   console.log("Suspès");  
7 }
```

Sentència if / else if / else

Permet encadenar **diverses condicions**:

```
1 let nota = 8;  
2  
3 if (nota < 5) {  
4   console.log("Suspès");
```

```
5 } else if (nota < 7) {  
6   console.log("Bé");  
7 } else if (nota < 9) {  
8   console.log("Notable");  
9 } else {  
10  console.log("Excel·lent");  
11 }
```

Es van comprovant les condicions en ordre fins que se'n compleixi una.

Sentència switch

És una alternativa **més llegible** a múltiples if / else if. Comprova un valor i executa el bloc corresponent al cas:

```
1 let color = "groc";  
2  
3 switch (color) {  
4   case "vermell":  
5     console.log("vermell");  
6     break;  
7   case "groc":  
8     console.log("groc");  
9     break;  
10  case "verd":  
11    console.log("verd");  
12    break;  
13  default:  
14    console.log("El color no és ni vermell, ni groc, ni verd");  
15 }
```

Cal utilitzar *break* per evitar que s'executin els casos següents.

1.3.6 Bucles (Iteracions)

Els bucles ens permeten **repetir un bloc de codi** mentre es compleixi una condició. Són molt útils per **recórrer arrays**, repetir càlculs o generar estructures dinàmiques.

Tots els bucles tenen habitualment tres parts:

1. **Inicialització:** defineix la variable de control.
2. **Condició de permanència:** s'avalua abans o després de cada iteració.
3. **Actualització:** modifica la variable de control per avançar en el procés.

while

El bucle while executa el codi mentre la condició sigui **certa**:

```
1 let i = 1;
2
3 while (i <= 5) {
4   console.log(i);
5   i++;
6 }
7 // Resultat: 1 2 3 4 5
```

do...while

Aquest bucle **garanteix que el bloc s'executa com a mínim una vegada**, ja que la condició es comprova **després**:

```
1 let i = 1;
2
3 do {
4   console.log(i);
5   i++;
6 } while (i <= 5);
7 // Resultat: 1 2 3 4 5
```

for

El bucle for és el més habitual. Reuneix les tres parts en una sola línia:

```
1 for (let i = 0; i < 5; i++) {
2   console.log(i);
3 }
4 // Resultat: 0 1 2 3 4
```

Bona pràctica: evitar càlculs innecessaris

Evita repetir `.length` en cada iteració:

```
1 let array = [1, 2, 3, 4];
2
3 // Forma poc òptima
4 for (let i = 0; i < array.length; i++) {
5   console.log(array[i]);
6 }
7
8 // Forma òptima
9 for (let i = 0, len = array.length; i < len; i++) {
10  console.log(array[i]);
11 }
```

forEach (només per arrays)

És un **mètode dels arrays** que executa una funció per a cada element:

```
1 let valors = [10, 20, 30];
2
3 valors.forEach(function(valor, index) {
4   console.log("A la posició " + index + " hi ha: " + valor);
5 });
```

for...in (per a objectes)

Aquest bucle recorre **les propietats d'un objecte**:

```
1 let cotxe = {  
2   marca: "Toyota",  
3   model: "Yaris",  
4   any: 2020  
5 };  
6  
7 for (var clau in cotxe) {  
8   console.log(clau + ": " + cotxe[clau]);  
9 }  
10 // marca: Toyota, model: Yaris, any: 2020
```

També existeix for...of, que recorre valors directament (millor per a arrays, cadenes i col·leccions iterables).

```
1 for (let valor of [10, 20, 30]) {  
2   console.log(valor); // 10 20 30  
3 }
```


2. Programació amb funcions

En la programació, les funcions són una de les eines més potents i fonamentals. Permeten organitzar el codi en blocs reutilitzables, facilitar la lectura i el manteniment dels programes, i reduir la repetició de codi. En JavaScript, les funcions són ciutadans de primera classe (first-class citizens), cosa que significa que poden ser tractades com qualsevol altra dada: es poden assignar a variables, passar com a paràmetres a altres funcions o fins i tot retornar-se com a resultat.

L'objectiu és entendre no només com es declaren i s'utilitzen les funcions, sinó també com pensar en termes de **modularitat i reutilització** —principis essencials en qualsevol llenguatge de programació modern.

2.1 Funcions predefinides del llenguatge

En tots els llenguatges de programació existeixen **llibreries de funcions** que serveixen per resoldre tasques habituals i repetitives. Aquestes llibreries estalvien als programadors haver d'escriure des de zero funcions comunes que sovint es necessiten en molts programes.

Un llenguatge de programació ben desenvolupat sol incloure **una bona quantitat de funcions i mètodes predefinits**, que faciliten molt la feina. De fet, en alguns casos pot ser més difícil **conèixer totes les llibreries** disponibles que aprendre la sintaxi bàsica del llenguatge.

En el cas de **JavaScript**, disposem de moltes funcions integrades. Com que és un llenguatge orientat a objectes, moltes d'aquestes funcions es troben **dins d'objectes** com Math (per operacions matemàtiques), String (per treballar amb cadenes), Array, Date, etc.

Tot i això, també hi ha **funcions globals** que no pertanyen a cap objecte concret i que podem utilitzar directament, com ara:

- `parseInt()`
- `parseFloat()`
- `isNaN()`
- `eval()`
- `alert()` o `console.log()`

Aquestes funcions **no necessiten conèixer cap objecte previ** i són especialment útils en les primeres etapes d'aprenentatge, ja que ens permeten **interactuar**

amb el llenguatge sense necessitat d'aprendre encara l'estructura completa dels objectes.

En aquesta lliçó ens centrarem justament en aquest tipus de funcions: les que **formen part del nucli de JavaScript** i es poden utilitzar de forma directa.

parseInt()

Converteix una cadena de text (String) en un **nombre enter**. Si la cadena comença amb caràcters no numèrics, retorna NaN.

```
1 parseInt("42");           // 42
2 parseInt("101", 2);       // 5 (base binària)
3 parseInt("3.14");         // 3
4 parseInt("abc");          // NaN
```

parseFloat()

Converteix una cadena de text en un **nombre decimal (coma flotant)**. També pot convertir nombres amb notació exponencial.

```
1 parseFloat("3.1416");     // 3.1416
2 parseFloat("10e2");        // 1000
3 parseFloat("5.5kg");       // 5.5
4 parseFloat("hola");        // NaN
```

isNaN()

Comprova si un valor **no és un número**. Retorna true si el valor és NaN, i false en cas contrari.

```
1 isNaN("hola");            // true (no és un número)
2 isNaN(5);                  // false
3 isNaN(NaN);                // true
4 isNaN("123");              // false (es pot convertir a número)
```

`isNaN()` fa una conversió prèvia a nombre. Si es vol evitar això, podem usar `Number.isNaN()` (més estricte).

eval()

Executa codi JavaScript que estigui contingut en una **cadena de text**. És molt potent, però perillós si no es controla bé (pot executar codi maliciós).

```
1 eval("2 + 2");             // 4
2 eval("var x = 5");         // defineix la variable x
3 console.log(x);            // 5
```

console.log()

Mostra informació a la **consola del navegador** o entorn d'execució. És molt útil per depurar (comprovar valors, traçar execució...).

```
1 let nom = "Joan";
2 console.log("Hola, " + nom); // Hola, Joan
```

alert()

Mostra una **finestra emergent (popup)** amb un missatge. S'utilitza per **avisos simples**, però no és recomanable en aplicacions modernes.

```
1 alert("Benvingut al curs de JavaScript!");
```

Quan s'executa, bloqueja l'execució del programa fins que l'usuari tanqui l'avís.

No volem donar una impressió equivocada amb aquesta petita llista de funcions natives de JavaScript. En realitat, existeixen moltes més funcions que anirem veient al llarg d'aquest material.

La diferència és que moltes d'aquestes funcions estan associades a objectes concrets. Per exemple:

- Les funcions per treballar amb **cadenes de caràcters** formen part de l'objecte `String`.
- Les funcions per fer **càlculs matemàtics avançats** estan dins de l'objecte `Math`.
- També hi ha funcions relacionades amb el **document HTML** (`document`) i amb la **finestra del navegador** (`window`).

A mesura que avancem en l'aprenentatge, **anirem descobrint aquestes funcions** i el seu ús pràctic dins del model d'objectes que ofereix JavaScript.

2.2 Funcions definides per l'usuari

Les **funcions en JavaScript** són blocs de codi executables als quals podem **passar paràmetres** i operar amb ells. Ens serveixen per **modularitzar els nostres programes** i estructurar-los en **blocs lògics que realitzin una tasca concreta**.

Això fa que el nostre codi sigui més **ordenat, llegible i fàcil de mantenir**, especialment quan el projecte creix.

2.2.1 Declaració de funció

Les funcions es declaren utilitzant la paraula reservada `function`, seguida habitualment d'un nom identificatiu per poder invocar-les més endavant en el codi.

Normalment, quan una funció finalitza la seva execució, retorna un valor mitjançant la paraula clau `return`. Aquest valor pot ser utilitzat per altres parts del programa.

```
1 function saludar(nom) {  
2   return "Hola, " + nom + "!";  
}
```

```
3 }  
4  
5 saludar("Joan"); // "Hola, Joan!"  
6 saludar("Marta"); // "Hola, Marta!"
```

Si no passem cap paràmetre:

```
1 saludar(); // "Hola, undefined!"
```

2.2.2 Funcions anònimes i assignació a variables

A JavaScript, podem crear **funcions sense nom** i assignar-les a una variable. Aquestes funcions s'anomenen **anònimes**, perquè no tenen identificador propi (no porten nom després de `function`).

Això ens permet **guardar la funció com si fos una dada més**, i després cridar-la utilitzant el nom de la variable.

```
1 // Assignem una funció anònima a la variable 'sumar'  
2 let sumar = function(a, b) {  
3   return a + b;  
4 };  
5  
6 // Ara podem cridar la funció com si fos una funció normal  
7 sumar(3, 4); // Retorna 7
```

2.2.3 Funcions fletxa (Arrow Functions)

Les **funcions fletxa** són una manera **més concisa** i moderna d'escriure funcions en JavaScript. Van ser introduïdes amb **ES6 (ECMAScript 2015)** i són molt utilitzades actualment, especialment en programació funcional i amb llibreries modernes com React o Vue.

Característiques principals:

- **Sintaxi més curta:** no cal escriure la paraula `function`.
- **Retorn implícit:** si el cos de la funció és una sola expressió, el resultat es retorna automàticament sense necessitat de `return`.
- **No tenen `this` propi:** hereten el `this` del context on han estat creades (important en funcions dins de mètodes o *callbacks*).
- No tenen arguments, `super` ni poden ser usades com a constructors (`new`).

Exemple bàsic:

```
1 const multiplicar = (a, b) => a * b;  
2  
3 multiplicar(3, 4); // 12
```

És equivalent a:

```
1 function multiplicar(a, b) {  
2   return a * b;  
3 }  
4  
5 multiplicar(3, 4); // 12
```

Quan només hi ha un paràmetre, podem **ometre els parèntesis**:

```
1 const doble = x => x * 2;
```

Equivalent a:

```
1 function doble(x) {  
2   return x * 2;  
3 }
```

Quan tenim zero, o més de dos paràmetres, cal mantenir els parèntesis:

```
1 const sumar = (a, b, c) => a + b + c;  
2 const mostrar = () => console.log("Hola món!");
```

Si la funció té més d'una línia o es vol fer servir return, cal obrir claus {}:

```
1 const mostrar = nom => {  
2   console.log("Hola " + nom);  
3   return true;  
4 };
```

Què entenem per "JavaScript modern"?

Quan parlem de **JavaScript modern**, ens referim a totes aquelles millores i noves funcionalitats que el llenguatge ha anat incorporant a partir de l'estàndard **ES6 (ECMAScript 2015)** i les versions posteriors. Aquestes novetats han fet que JavaScript sigui més potent, més fàcil d'escriure i més proper als llenguatges de programació actuals.

Alguns exemples d'aquest "JavaScript modern" són:

- **Funcions fletxa (arrow functions)** → sintaxi més curta i clara.
- **let i const** → millor control de l'abast (*scope*) de les variables.
- **Templates literals** → permeten inserir variables dins de cadenes amb `${}`.
- **Classes i mòduls** → per organitzar millor el codi i reutilitzar-lo.
- **Promeses i async/await** → nova manera d'abordar la programació asíncrona.

2.2.4 Paràmetres

Podem assignar valors per defecte en cas que no es passin arguments:

```
1 function saludar(nom = "amic") {  
2   return 'Hola, ${nom}';  
3 }  
4  
5 saludar(); // "Hola, amic"  
6 saludar("Pau"); // "Hola, Pau"
```

Encara que no especifiquem paràmetres, podem accedir als arguments passats amb arguments:

```
1 function llistar() {  
2   for (var i = 0; i < arguments.length; i++) {  
3     console.log(arguments[i]);  
4   }  
5 }  
6  
7 llistar("a", "b", "c");  
8 // Mostra: a, b, c
```

arguments només funciona
amb funcions normals, no
amb funcions fletxa.

2.2.5 Àmbit (Scope) de les variables

L'**àmbit (scope)** determina **on és visible una variable** dins del codi. Un bloc és qualsevol secció de codi delimitada per claus {}, com per exemple dins d'un if, for, while, function, etc. Les variables declarades dins d'una funció només són visibles en aquell entorn (àmbit local). Les de fora són globals.

Quan declarem una variable amb var, és **visible a tota la funció (àmbit global)** on es troba, encara que la definim dins d'un bloc. Això pot provocar errors difícils de detectar i és per això que no es recomana el seu ús.

Exemple:

```
1 function prova() {  
2   if (true) {  
3     var x = 5;  
4   }  
5   console.log(x); // 5 (tot i estar fora del bloc if)  
6 }
```

Les variables declarades amb let o const **només existeixen dins del bloc (àmbit local)** on han estat definides. Això evita errors per redefinicions accidentals o accessos indeguts.

Exemple amb let:

```
1 function prova() {
```

```
2   if (true) {  
3       let x = 5;  
4   }  
5   console.log(x); // Error: x is not defined  
6 }
```

Exemple amb const:

```
1 {  
2     const pi = 3.14;  
3 }  
4  
5 console.log(pi); // Error: pi is not defined
```

2.3 Conceptes avançats de funcions

En aquesta secció veurem com les funcions a JavaScript van molt més enllà de simples blocs de codi. Poden guardar informació del seu context (*closures*), actuar com a objectes de primera classe i fins i tot servir per crear altres funcions o objectes. També entendrem millor el paper del `this`, un concepte clau per treballar amb el context d'execució.

2.3.1 Closures (funcions tancades)

Un **closure** és una funció **que s'emporta amb ella l'entorn on va ser creada**, és a dir, **recorda les variables locals de la seva funció mare**, fins i tot després que aquesta hagi acabat d'executar-se.

Això permet:

- Encapsular dades privades que no es poden accedir directament des de fora.
- Crear funcions personalitzades amb valors fixats (com en configuradors, factories, etc.).

Anem a veure un exemple:

```
1 function crearComptador() {  
2     let comptador = 0;  
3  
4     return function() {  
5         comptador++;  
6         return comptador;  
7     };  
8 }  
9  
10 const cont1 = crearComptador();  
11 console.log(cont1()); // 1
```

```
12 console.log(cont1()); // 2
13
14 const cont2 = crearComptador();
15 console.log(cont2()); // 1
```

Cada `crearComptador()` té la seva pròpia còpia privada de comptador, que no és compartida amb altres instàncies. Fixem-nos que posem `cont1()` i no `cont1` perquè la funció `crearComptador()` retorna una funció.

2.3.2 Funcions com a objectes

A JavaScript, **les funcions són tractades com valors**, igual que els números, cadenes o arrays. Això vol dir que **una funció és un objecte de primera classe** (*first-class object*), cosa que ens permet:

- Assignar funcions a variables
- Passar funcions com a arguments a altres funcions
- Retornar funcions com a resultat
- Guardar funcions dins d'estructures de dades (arrays, objectes...)

Això obre la porta a una programació molt més flexible i potent, com per exemple:

- *Callbacks*
- *Closures*
- Funcions de més alt nivell
- Programació funcional

Què és la programació funcional?

La programació funcional és un estil de programació que tracta les **funcions com a primers ciutadans** (*first-class citizens*). Això vol dir que les funcions no només serveixen per executar codi, sinó que també es poden **guardar en variables, passar com a arguments i retornar com a resultat** d'altres funcions.

Aquest enfocament permet escriure programes més **modulars, reutilitzables i fàcils de provar**, ja que cada funció és independent i no depèn de l'estat global.

Exemples típics en JavaScript:

- Passar una funció a `map()`, `filter()` o `forEach()`.
- Retornar funcions especialitzades mitjançant *closures*.
- Crear arrays d'operacions que podem executar dinàmicament.

Assignar funcions a variables:

```
1 const sumar = function(a, b) {  
2   return a + b;  
3 };  
4  
5 console.log(sumar(2, 5)); // 7  
6 typeof(sumar); // function
```

Funció com a paràmetre:

```
1 function operar(op) {  
2   return op(3, 4); // executem la funció que rebem com a parà  
3     metre  
4 }  
5 // Passem una funció fletxa com a argument  
6 operar((a, b) => a * b); // 12  
7  
8 // També podem utilitzar una funció tradicional  
9 operar(function(a, b) {  
10   return a * b;  
11 });
```

Aquí la funció `operar` rep una altra funció com a paràmetre (`op`) i la crida internament amb dos valors. Aquest patró és molt comú en *callbacks* i *frameworks* moderns.

Retornar funcions:

```
1 function crearOperadorMultiplicacio(x) {  
2   return function(y) {  
3     return x * y;  
4   };  
5 }  
6  
7 const perTres = crearOperadorMultiplicacio(3);  
8 console.log(perTres(4)); // 12
```

En aquest cas, estem **retornant una funció que recorda el valor de x** (*closure*), creant una funció especialitzada en multiplicar per 3.

Guardar funcions dins d'un array:

```
1 const operacions = [  
2   (a, b) => a + b,  
3   (a, b) => a - b,  
4   (a, b) => a * b,  
5   (a, b) => a / b,  
6 ];  
7  
8 console.log(operacions[2](6, 2)); // 12 (multiplicació)  
9 // Accedim al tercer element de l'array
```

Les funcions també tenen propietats

Les funcions són objectes especials i poden tenir **propietats pròpies**:

Callback

Anomenem *callback* a una funció que passem com a paràmetre a una altra funció perquè s'executi més endavant, quan sigui necessari. S'utilitza molt en JavaScript per gestionar accions que triguen temps (com llegir dades, esperar un esdeveniment o treballar amb temporitzadors).

```
1 function saluda(nom) {
2   return "Hola " + nom;
3 }
4
5 saluda.descripcion = "Funció que saluda";
6
7 console.log(saluda.descripcion); // "Funció que saluda"
```

2.3.3 Funcions com a fàbriques (closures + mètodes)

Una **fàbrica de funcions** o **factory function** és una funció que crea i retorna un objecte amb mètodes propis. Aquesta tècnica permet **encapsular dades** mitjançant *closures* i retornar funcions que poden accedir-hi de manera segura.

Aquesta estratègia s'utilitza quan volem:

- Crear **objectes personalitzats** sense fer servir class.
- Mantenir **valors interns privats**.
- Exposar només una **interfície controlada** (funcions públiques).

Anem a veure un exemple on tindrem una variable **privada** (n) que només serà accessible dins de la funció comptador:

```
1 function comptador() {
2   let n = 0; // variable privada
3
4   return {
5     incrementar: () => ++n, // incrementa i retorna el nou
      valor
6     decrementar: () => --n, // decrementa i retorna el nou
      valor
7     valor: () => n          // retorna el valor actual
8   };
9 }
```

Cada mètode és una **funció fletxa que forma part d'un closure**, i pot accedir a la variable `n` gràcies al context tancat.

Ús del comptador:

```
1 const c = comptador();
2
3 c.incrementar(); // 1
4 c.incrementar(); // 2
5 c.valor();       // 2
6
7 c.decrementar(); // 1
8
9 // La variable n no és accessible des de fora
10 console.log(c.n); // undefined
```


Això garanteix que **només es pot modificar n a través dels mètodes que hem dissenyat**, una manera molt neta d'encapsular dades.

Els avantatges d'aquest patró de disseny són:

- **Seguretat**: no exposem dades directament, només funcions controlades.
- **Reutilització**: podem crear múltiples instàncies independents.

```
1 const a = comptador();
2 const b = comptador();
3
4 a.incrementar(); // 1
5 b.incrementar(); // 1
6 a.incrementar(); // 2
7 b.valor();       // 1
```

Cada instància té **la seva pròpia còpia privada** de n.

2.3.4 El context "this"

En JavaScript, **this** és una **paraula clau** que fa referència al **context d'execució actual**, és a dir, **l'objecte que *posseix* la funció** quan s'executa.

Però **el valor de this canvia segons com s'invoca la funció**, i això pot ser confús si no ho coneixem bé.

Les funcions fletxa **no tenen el seu propi this**, sinó que prenen el **this** de l'àmbit on han estat creades. Això pot evitar errors en certs contextos com en *callbacks* dins d'objectes:

```
1 function Persona(nom) {
2   this.nom = nom;
3   setTimeout(() => {
4     console.log("Hola, soc " + this.nom); // correcte: this fa
5     // referència a Persona
6   }, 1000);
7 }
```

En canvi, amb la **function clàssica** no funcionaria igual:

```
1 function Persona(nom) {
2   this.nom = nom;
3   setTimeout(function() {
4     console.log("Hola, soc " + this.nom); // incorrecte: this no
5     // apunta a Persona
6   }, 1000);
7 }
```

En els exemples anteriors les funcions fletxa ens han anat bé perquè volíem accedir a una propietat que estava al mateix nivell de l'àmbit on havia estat creada.

Però a vegades ens pot jugar males passades. Fixem-nos en aquest exemple:

```
1 const obj = {  
2   nom: "Marc",  
3   normal: function () {  
4     console.log(this.nom); // "Marc"  
5   },  
6   fletxa: () => {  
7     console.log(this.nom); // undefined (this apunta a context  
8     global)  
9   },  
10 };  
11 obj.normal(); // "Marc"  
12 obj.fletxa(); // undefined
```

Quan cridem `obj.normal()`, la funció s'executa **com a mètode de l'objecte `obj`**, i per tant **`this` apunta a `obj`**. Això és el comportament normal que esperem.

Les **funcions fletxa no tenen `this` propi**. Quan les definim, **hereten el `this` del context en què van ser creades**.

La funció fletxa **no hereta el `this` de l'objecte `obj`**, perquè **l'objecte literal no crea un nou àmbit de `this`**. Això vol dir que `this` apunta **al context exterior**, que és el **context global** (o `undefined` en `use strict`).

Per tant, quan fem `obj.fletxa()`, el `this` no és `obj`, sinó el `this` global.

Quan convé fer servir arrow functions?

- Quan necessitem una funció curta i clara.
- Quan volem heretar el `this` del context actual.
- En *callbacks*, com `.map()`, `.filter()`, `.forEach()`, etc. (els veurem més endavant).

Quan no convé usar-les:

- Quan necessitem accedir a `this` d'un objecte creat amb `function` (ex. mètodes)
- Quan volem una funció constructora amb `new` (no és possible)

3. Programació amb arrays i objectes

En aquesta lliçó explorarem com JavaScript permet treballar amb dades estructurades i comportaments associats mitjançant objectes. Aprendre a crear i gestionar **arrays** i **objectes** definits pel programador, i introduïrem els principis bàsics de la programació orientada a objectes (POO), com ara classes, propietats i mètodes. Finalment, veurem alguns exemples pràctics que mostren com aplicar aquests conceptes per escriure codi més modular, reutilitzable i fàcil de mantenir.

3.1 Arrays. Objectes definits pel programador

En aquest apartat veurem com JavaScript ens permet organitzar i gestionar dades de manera eficient mitjançant arrays i objectes. Els arrays són col·leccions ordenades d'elements, ideals per emmagatzemar llistes o conjunts de valors, mentre que els objectes ens permeten agrupar informació relacionada en una mateixa estructura amb claus i valors. A més, combinant aquests elements, podem crear estructures de dades complexes adaptades a les necessitats del nostre programa.

3.1.1 Arrays com estructures de dades

Un **array** és una estructura que ens permet **emmagatzemar diversos valors en una sola variable**, de forma **ordenada**. És molt útil quan volem treballar amb **llistes de dades**, com noms, números, objectes, etc.

Els arrays en JavaScript poden contenir **qualsevol mena de dada**, fins i tot barrejats (nombres, *strings*, booleans, objectes, altres arrays...).

Creació d'un array

A continuació, veurem alguns exemples de com crear arrays.

```
1 let buida = []; // Array buit
2 let numeros = [1, 2, 3, 4]; // Array numèric
3 let noms = ["Anna", "Pau", "Joan"]; // Array d'Strings
4 let mesclat = [1, "hola", true, [10, 20]]; // Array mixt
```

També es pot fer amb el constructor:

```
1 let colors = new Array("vermell", "blau", "verd");
```

Tot i això, la forma recomanada és la de claudàtors ([]), per ser més clara i segura.

Accés als elements

Els elements es compten des de l'índex zero (0 és el primer element).

```
1 let numeros = [10, 20, 30];
2 console.log(numeros[0]); // 10
3 console.log(numeros[2]); // 30
```

Longitud de l'array

Cada array té una propietat `.length` que indica quants elements conté:

```
1 numeros.length; // 3
```

Aquesta propietat també es pot usar per recórrer l'array amb un bucle:

```
1 for (let i = 0; i < numeros.length; i++) {
2   console.log(numeros[i]);
3 }
```

Mètodes útils dels arrays

Els arrays disposen de molts **mètodes integrats** per facilitar les operacions habituals. A continuació veurem un llistat dels més usuals:

TAULA 3.1. Mètodes útils dels arrays

Mètode	Què fa
<code>push()</code>	Afegeix un element al final
<code>pop()</code>	Elimina i retorna l'últim element
<code>unshift()</code>	Afegeix un element al principi
<code>shift()</code>	Elimina i retorna el primer element
<code>join(separador)</code>	Uneix tots els elements en un <i>string</i> , separats per separador. El separador per defecte és la coma ,
<code>concat()</code>	Uneix dos arrays en un de sol
<code>slice(inici, fi)</code>	Retorna una part de l'array (no modifica l'original)
<code>splice()</code>	Afegeix/elimina elements de l'array (sí que modifica l'original)
<code>indexOf(elem)</code>	Retorna la posició d'un element (o -1 si no hi és)
<code>includes(elem)</code>	Indica si un element es troba dins l'array

Anem a veure alguns exemples del mètodes anteriors:

```
1 let fruites = ["poma", "plàtan", "kiwi"];
2
3 fruites.push("maduixa");           // ["poma", "plàtan", "kiwi", "
   maduixa"]
```

```
4 fruites.shift();           // ["plàtan", "kiwi", "maduixa"]
5 fruites.includes("kiwi");   // true
6 fruites.indexOf("plàtan");  // 0 (Ha canviat després d'
    aplicar el mètode shift)
```

Iterar amb ".forEach()"

El mètode `.forEach()` ens permet **recórrer tots els elements d'un array** i executar una funció per a cadascun d'ells. És una alternativa moderna i més llegible que els bucles `for`.

No retorna cap nou array: simplement **executa una acció per cada element**.

```
1 let animals = ["gat", "gos", "ocell"];
2
3 animals.forEach(function(animal, index) {
4     console.log(index + ": " + animal);
5 });
```

També es pot fer amb funcions fletxa:

```
1 animals.forEach((animal, i) => console.log(i + ": " + animal));
```

Transformar arrays: ".map()" i ".filter()"

Els mètodes `.map()` i `.filter()` ens permeten crear un nou array a partir de l'original:

- `map()` transforma cada element i retorna un array nou amb els resultats.
- `filter()` selecciona només els elements que compleixen una condició.

Aquests mètodes no modifiquen l'array original.

```
1 let nums = [1, 2, 3, 4];
2
3 // Multiplica per 2 cada element
4 let dobles = nums.map(n => n * 2); // [2, 4, 6, 8]
5
6 // Filtra els nombres parells
7 let parells = nums.filter(n => n % 2 === 0); // [2, 4]
```

3.1.2 Objectes com estructures de dades

A JavaScript, un **objecte** és una estructura que ens permet **emmagatzemar dades en forma de clau-valor**. És una eina fonamental per representar entitats més complexes i flexibles que un array, com ara una persona, un cotxe, un usuari, etc.

Mentre que els arrays organitzen les dades per **posició numèrica (índexs)**, els objectes ho fan per **claus amb nom (strings)**.

Creació d'un objecte

A continuació, es mostra com crear un **objecte literal**. Un objecte literal en JavaScript és una forma directa i senzilla de crear un objecte amb *claus* (propietats) i *valors*.

```
1 let persona = {  
2   nom: "Anna",  
3   edat: 28,  
4   viu: true  
5 };
```

Cada propietat té una *clau* (ex. “nom”) i un *valor* associat (ex. “Anna”).

Accés a propietats

Podem accedir a les propietats amb notació de punt o notació de claudàtors:

```
1 // Punt (dot notation)  
2 console.log(persona.nom); // "Anna"  
3  
4 // Claudàtors (bracket notation)  
5 console.log(persona["edat"]); // 28
```

Notació amb claudàtors

La notació amb claudàtors és útil quan la clau és una variable o conté espais:

```
1 let clau = "viu";  
2 console.log(persona[clau]); // true
```

Modificar i afegir propietats

Els objectes en JavaScript són **dinàmics**, la qual cosa vol dir que podem afegir noves propietats o modificar-ne les existents en qualsevol moment, fins i tot després d'haver estat creats.

Aquesta flexibilitat fa que els objectes siguin molt útils com a contenidors d'informació estructurada.

```
1 persona.edat = 29; // Modifiquem  
2 persona.professio = "dissenyadora"; // Afegim nova propietat
```

Mètodes dins d'un objecte literal

A més de propietats amb valors com números o textos, un objecte literal també pot incloure **funcions**. Quan una funció forma part d'un objecte, l'anomenem **mètode**.

Aquests mètodes permeten definir accions o comportaments relacionats amb l'objecte.

```
1 let persona = {  
2   nom: "Anna",  
3   edat: 28,  
4   viu: true,  
5   saludar() {  
6     console.log('Hola! Soc ${this.nom}.');  
7   }  
8 };  
9  
10 persona.saludar(); // "Hola! Soc Anna."
```

Els mètodes poden accedir a les propietats de l'objecte mitjançant la paraula clau `this`, que fa referència a l'objecte mateix.

Eliminar propietats

La paraula clau `delete` serveix per **eliminar propietats d'un objecte**. Quan s'utilitza, **esborra la clau i el valor** associat, de manera que ja no existeixen dins de l'objecte.

```
1 delete persona.viu;
```

També podem eliminar **mètodes** (funcions dins d'un objecte), ja que internament són propietats com qualsevol altra.

```
1 let persona = {  
2   nom: "Anna",  
3   edat: 29,  
4   saludar() {  
5     console.log('Hola, soc ${this.nom}');  
6   }  
7 };  
8  
9 persona.saludar(); // "Hola, soc Anna"  
10 delete persona.saludar; // Eliminem el mètode  
11 console.log(persona.saludar); // undefined
```

Després d'eliminar-lo, el mètode ja no està disponible dins de l'objecte.

Recórrer les propietats amb "for...in"

Quan treballem amb objectes, sovint volem **recórrer totes les seves propietats** per llegir o processar els valors que contenen. En JavaScript, això es pot fer fàcilment amb el bucle especial **for...in**.

Aquest bucle ens permet **iterar per totes les claus (propietats enumerables)** d'un objecte.

```
1 for (let clau in persona) {  
2   console.log(clau + ": " + persona[clau]);  
}
```

```
3 }  
4 // nom: Anna  
5 // edat: 29  
6 // professio: dissenyadora
```

Els objectes són especialment útils quan necessitem **representar entitats complexes** que tenen diverses característiques o propietats. Per exemple, si volem descriure un usuari, un llibre o un producte, cadascun d'aquests elements tindrà múltiples dades associades, com el nom, l'edat, el preu o la categoria. En aquests casos, un array no seria suficient, ja que només ens permet emmagatzemar valors de manera ordenada, però **sense significat semàntic clar**.

A diferència dels arrays, que organitzen la informació per posició (índex), els objectes ens permeten accedir a cada dada mitjançant **una clau descriptiva**, com nom, edat o preu. Això fa que el codi sigui més **llegible i entenedor**, sobretot quan treballem amb dades estructurades.

En resum, utilitzarem objectes sempre que vulguem **estructurar informació amb sentit** (estructures de dades), i no només llistes de valors. Són una eina fonamental per construir programes clars, escalables i fàcils de mantenir.

3.1.3 Estructures de dades complexes

Quan treballem amb dades reals en una aplicació, sovint no n'hi ha prou amb estructures simples com arrays o objectes independents. En aquests casos, JavaScript ens permet **combinar arrays i objectes** per construir **estructures de dades complexes**, molt més potents i representatives del món real.

Arrays que contenen objectes

Una de les combinacions més habituals és tenir un **array d'objectes**. Aquesta estructura és ideal quan volem representar una col·lecció d'elements que comparteixen una mateixa estructura, com ara una llista de llibres, d'usuaris o de productes. Cada objecte de l'array representa un element amb les seves pròpies propietats.

Per exemple, imaginem que volem representar un catàleg de llibres:

```
1 let llibres = [  
2   { titol: "1984", autor: "George Orwell", any: 1949 },  
3   { titol: "Crim i càstig", autor: "Fiódor Dostoievski", any:  
4     1866 },  
5   { titol: "L'ombra del vent", autor: "Carlos Ruiz Zafón", any:  
     2001 }  
];
```

En aquest cas, l'array `llibres` conté tres objectes, cadascun amb informació rellevant sobre un llibre. Això ens permet, per exemple, recórrer tota la llista

amb un bucle i mostrar només els títols o buscar l'autor d'un llibre concret.

Aquesta estructura és especialment útil quan gestionem dades provinents d'una API o una base de dades, ja que aquestes normalment retornen col·leccions d'elements representats com objectes dins d'un array.

Objectes amb propietats que són arrays

També és molt comú el cas invers: **objectes que contenen arrays com a valors d'algunes propietats**. Això ens permet representar característiques que poden tenir **múltiples valors associats**, com per exemple les aficions d'una persona, les notes d'un alumne, o les etiquetes d'un producte.

Observem aquest exemple:

```
1 let usuari = {  
2   nom: "Marta",  
3   edat: 22,  
4   aficions: ["lectura", "escalada", "viatjar"]  
5 };
```

Aquí l'objecte usuari conté una propietat aficions que és un array. Això ens permet treballar de manera eficient amb múltiples valors relacionats amb un sol camp, mantenint una estructura lògica i clara.

Podem accedir-hi i treballar-hi fàcilment:

```
1 console.log(usuari.aficions[1]); // "escalada"  
2 usuari.aficions.push("cinema");
```

Aquest tipus de combinacions ens permeten **modelar la realitat** dins del nostre programa. A diferència d'un full de càlcul o una simple llista de valors, aquí podem representar relacions complexes, jerarquies i col·leccions d'informació estructurada.

És gràcies a aquestes estructures que podem construir funcionalitats com:

- Llistes de tasques amb subtasques
- Perfils d'usuari amb múltiples preferències
- Carretons de compra amb productes i quantitats
- Aplicacions de calendari amb esdeveniments per dies

Dominar les estructures de dades complexes ens permet construir aplicacions **més potents i realistes**. JavaScript ens dona la flexibilitat de **combinar arrays i objectes lliurement**, cosa que és fonamental per treballar amb dades de forma organitzada, neta i escalable.

3.2 Programació orientada a objectes

JavaScript és un llenguatge molt flexible i dinàmic, i una de les seves característiques més potents és que permet programar seguint el paradigma orientat a objectes. Això vol dir que podem organitzar el nostre codi en estructures que encapsulen dades i comportaments, facilitant la reutilització, la modularitat i la comprensió del programa.

Tot i que internament JavaScript es basa en un sistema d'**herència per prototips**, des de la versió **ES6 (2015)** s'introdueixen les **classes**, que ens permeten escriure codi amb una sintaxi més familiar i clara, especialment per a persones acostumades a llenguatges com Java, Python o C++.

La programació orientada a objectes amb JavaScript ens permet **modelar el món real** a través de classes, objectes, propietats i mètodes. Gràcies a la sintaxi moderna de classes, escriure codi estructurat i reutilitzable és molt més senzill i intuïtiu.

Aquest estil de programació és fonamental quan treballem en projectes grans, interfícies complexes o aplicacions que gestionen molts tipus d'entitats i relacions. Aprendre a utilitzar correctament les classes, l'herència i l'encapsulació ens dona una eina potent per escriure **codi net, mantenible i escalable**.

3.2.1 Introducció a les classes

Una classe és un esquema o plantilla que defineix com han de ser els objectes que es creïn a partir d'ella. Ens permet agrupar les propietats i els mètodes que definiran el comportament d'una entitat concreta, com ara un usuari, una mascota, una factura o qualsevol element que volem modelar.

En JavaScript, podem definir una classe utilitzant la paraula clau `class`, i afegir-hi un constructor per inicialitzar les propietats. A diferència d'altres llenguatges, no cal declarar els atributs fora del constructor.

Per exemple, si volem definir una classe `Persona`, podríem escriure el següent:

```
1 class Persona {  
2   constructor(nom, edat) {  
3     this.nom = nom;  
4     this.edat = edat;  
5   }  
6  
7   saludar() {  
8     console.log(`Hola, soc ${this.nom} i tinc ${this.edat} anys`  
9       .');  
10  }  
11 }
```

Quan fem servir el mot clau `new`, estem creant una nova instància d'aquesta classe. Això vol dir que podem crear tantes persones com vulguem, totes amb el seu nom i edat particulars, però compartint el comportament comú definit dins la classe.

```
1 const p1 = new Persona("Anna", 30);
2 const p2 = new Persona("Joan", 22);
3
4 p1.saludar(); // Hola, soc Anna i tinc 30 anys.
5 p2.saludar(); // Hola, soc Joan i tinc 22 anys.
```

3.2.2 El paper de 'this' dins de les classes

Dins d'una classe, la paraula clau `this` fa referència a la instància concreta de l'objecte. És a dir, quan escrivim `this.nom`, estem accedint a la propietat `nom` de l'objecte que s'utilitza en aquell moment. Aquest comportament és essencial per distingir entre els valors que rep el constructor i els que es guarden dins l'objecte.

3.2.3 Encapsulació i modificació de valors

A diferència de llenguatges fortament orientats a objectes, JavaScript no obliga a declarar les propietats com a públiques o privades. Però a partir d'ES2022 ja és possible usar el símbol `#` per definir **propietats privades**, que només poden ser accedides des de dins de la classe.

```
1 class Compte {
2   #saldo;
3
4   constructor(titular) {
5     this.titular = titular;
6     this.#saldo = 0;
7   }
8
9   ingressar(quantitat) {
10    this.#saldo += quantitat;
11  }
12
13  veureSaldo() {
14    return this.#saldo;
15  }
16 }
17
18 const c = new Compte("Laura");
19 c.ingressar(100);
20 console.log(c.veureSaldo()); // 100
21 console.log(c.#saldo);       // Error: propietat privada
```

Aquest sistema ens ajuda a protegir les dades internes i evitar que es modifiquin des de fora de la classe, millorant la seguretat i robustesa del codi.

Propietats públiques vs. privades

JavaScript **obliga** a declarar les propietats privades amb # **a dalt de tot dins de la classe**, abans de fer-les servir dins del constructor o de qualsevol mètode. Això forma part de l'especificació del llenguatge.

Per contra, les propietats **públiques** (com `this.titular`) **no cal declarar-les prèviament**. El fet de fer `this.titular = ...` dins del constructor **ja crea automàticament la propietat a l'objecte**.

Ara bé, sovint volem **controlar encara més l'accés o la modificació de certs valors**, però sense haver d'utilitzar mètodes com `veureSaldo()` o `modificarSaldo()`, que poden resultar menys intuïtius. En aquests casos, JavaScript ens ofereix una eina molt útil: els **accessors de dades** o **data accessors**, que s'implementen mitjançant les paraules clau `get` i `set`.

Els accessors ens permeten definir **com es llegeix (get) o com s'escriu (set) una propietat**, però des del punt de vista extern, l'ús serà igual que si accedíssim directament a una propietat. Això fa que el codi sigui més net i expressiu, i alhora ens permet **validar o transformar les dades en segon pla**.

Per exemple, podem reescriure el cas anterior del `Compte` per fer servir `get` i `set` en lloc de mètodes tradicionals:

```
1 class Compte {
2   #saldo;
3
4   constructor(titular) {
5     this.titular = titular;
6     this.#saldo = 0;
7   }
8
9   get saldo() {
10    return this.#saldo.toFixed(2) + " €";
11  }
12
13  set saldo(quantitat) {
14    if (quantitat >= 0) {
15      this.#saldo = quantitat;
16    } else {
17      console.log("No es pot assignar un saldo negatiu.");
18    }
19  }
20 }
```

Ara, l'ús extern és molt més natural:

```
1 const compte = new Compte("Laura");
2
3 compte.saldo = 200;           // assignació controlada amb el
   set
4 console.log(compte.saldo);    // lectura controlada amb el get
   → "200.00 €"
5
6 compte.saldo = -50;          // no es permet, salta l'avís
```

Amb aquesta estructura, mantenim la protecció de la propietat interna `#saldo`, però oferim **una interfície d'accés elegant i segura**, molt semblant a les propietats públiques d'altres llenguatges orientats a objectes.

Els *data accessors* són especialment útils quan volem que **el comportament d'una propietat sembli directe, però internament amagui lògica de control o format**, i per tant constitueixen una extensió molt natural de l'encapsulació moderna en JavaScript.

3.2.4 Abstracció

L'**abstracció** és un dels pilars fonamentals de la programació orientada a objectes. Consisteix a **ocultar els detalls interns d'una implementació i mostrar només la interfície necessària** per a qui l'utilitza. L'objectiu és **simplificar la complexitat**, evitant que el codi que utilitza un objecte hagi de conèixer com funciona per dins.

En llenguatges com **Java**, l'abstracció es formalitza mitjançant **classes abstractes i interfaces**, que permeten definir què pot fer un objecte sense dir exactament com ho fa. Això facilita la separació entre definició i implementació.

En canvi, JavaScript no té classes abstractes com a construcció formal, però això no vol dir que no pugui implementar l'abstracció. En JavaScript, podem aconseguir l'abstracció encapsulant detalls interns, fent servir propietats privades (amb #) i exposant només mètodes públics per interactuar amb l'objecte.

Exemple d'abstracció en JavaScript:

```
1 class Sensor {  
2   #valorIntern;  
3  
4   constructor() {  
5     this.#valorIntern = Math.random() * 100;  
6   }  
7  
8   obtenirLectura() {  
9     return this.#valorIntern.toFixed(2) + " u.";  
10  }  
11 }
```

En aquest cas, el codi que utilitza el sensor només pot fer:

```
1 const s = new Sensor();  
2 console.log(s.obtenirLectura()); // "47.39 u." (per exemple)
```

No pot accedir a #valorIntern ni modificar-lo directament, perquè forma part de la implementació interna. L'usuari només veu el mètode públic obtenirLectura(), que és tot el que necessita per fer servir correctament l'objecte.

Comparació amb un exemple en Java:

```
1 public abstract class Sensor {  
2   protected double valorIntern;  
3  
4   public Sensor() {  
5     this.valorIntern = Math.random() * 100;  
6   }  
7 }
```

```
7
8     public abstract String obtenirLectura();
9 }
10
11 public class SensorTemperatura extends Sensor {
12     @Override
13     public String obtenirLectura() {
14         return String.format("%.2f ºC", valorIntern);
15     }
16 }
```

I el seu ús seria:

```
1 Sensor s = new SensorTemperatura();
2 System.out.println(s.obtenirLectura());
```

En aquest cas, la **classe abstracta Sensor** defineix **què ha de fer** qualsevol sensor (`obtenirLectura()`), però no com. La classe *filla* (`SensorTemperatura`) implementa el comportament concret. Això és **abstracció formal**, amb suport explícit del llenguatge.

3.2.5 Herència entre classes

La programació orientada a objectes també permet **heretar comportaments** d'una classe base. Això vol dir que podem definir una classe general i després crear altres classes més específiques que parteixen de la primera i afegeixen funcionalitats pròpies.

Per fer-ho, JavaScript ens ofereix les paraules clau `extends` i `super`:

```
1 class Animal {
2     constructor(nom) {
3         this.nom = nom;
4     }
5
6     parlar() {
7         console.log(`${this.nom} fa un so.`);
8     }
9 }
10
11 class Gos extends Animal {
12     parlar() {
13         console.log(`${this.nom} borda.`);
14     }
15 }
16
17 const layca = new Animal("Layca");
18 layca.parlar(); // Layca fa un so.
19 const rex = new Gos("Rex");
20 rex.parlar(); // Rex borda.
```

Quan una classe hereta d'una altra, pot **sobreescriure mètodes** per adaptar-ne el comportament, com hem vist amb el mètode `parlar()`. Aquesta sobreescritura de mètodes és coneix com a **polimorfisme**.

3.2.6 Polimorfisme

El **polimorfisme** ens permet definir mètodes amb el mateix nom però comportaments diferents, segons el tipus d'objecte que els implementa. En JavaScript, això es pot aconseguir mitjançant sobreescritura de mètodes en classes filles.

```
1 class Empleat {
2   saludar() {
3     console.log("Hola, soc un empleat.");
4   }
5 }
6
7 class Cap extends Empleat {
8   saludar() {
9     console.log("Hola, soc el cap!");
10  }
11 }
12
13 const e = new Empleat();
14 const c = new Cap();
15
16 e.saludar(); // "Hola, soc un empleat."
17 c.saludar(); // "Hola, soc el cap!"
```

Tots dos objectes tenen el mètode `saludar()`, però el comportament és diferent: això és **polimorfisme**.

3.2.7 Una menció als prototips

Tot i que actualment la majoria de programadors fan servir la sintaxi de classes, és important saber que **les classes són només una forma més clara de treballar amb els prototips**, que són el veritable sistema d'herència de JavaScript. Cada objecte en JavaScript té un enllaç intern al seu prototip, i si no troba una propietat o mètode en ell mateix, la busca al prototip del qual deriva.

Aquesta cadena d'herència es coneix com a **cadena de prototips**. Tot i això, gràcies a les classes, ja no cal definir manualment mètodes al prototip, cosa que abans es feia així:

```
1 function Persona(nom) {
2   this.nom = nom;
3 }
4
5 Persona.prototype.saludar = function() {
6   console.log('Hola, soc ${this.nom}');
7 };
```

Aquest codi funciona, però és molt menys llegible que la versió moderna amb `class`.

3.3 Exemples pràctics

Després de tota la teoria que hem treballat sobre programació orientada a objectes en JavaScript —incloent l'ús de classes, encapsulació, herència, polimorfisme, abstracció i accessors de dades— és fonamental consolidar aquests coneixements mitjançant exemples pràctics. Això ajuda a visualitzar com aplicar aquests conceptes a situacions reals i fa que entenguem no només la sintaxi, sinó també **l'estructura i la intenció del codi**.

3.3.1 Exemple 1: Estudiants i qualificacions

Es demana crear una petita aplicació per **gestionar les qualificacions d'estudiants** en diferents assignatures.

Cada estudiant té un **nom** i un **DNI** (que ha de ser privat). A més, pot estar matriculat en diverses assignatures, cadascuna amb una **llista de notes**.

Cada assignatura ha de permetre:

- Afegir notes vàlides (entre 0 i 10).
- Calcular automàticament la **mitjana** de les notes amb un accés get.

Cada estudiant ha de poder:

- Afegir assignatures al seu historial.
- Consultar la seva identificació (nom + DNI).
- Mostrar un llistat de totes les assignatures amb la seva mitjana.

Per reforçar la **encapsulació i l'abstracció**, es demana que la informació privada no sigui accessible directament, i que s'ofereixin **mètodes públics** per interactuar amb l'objecte de forma segura.

```
1 class Assignatura {
2   constructor(nom) {
3     this.nom = nom;
4     this.notes = [];
5   }
6
7   afegirNota(nota) {
8     if (nota >= 0 && nota <= 10) {
9       this.notes.push(nota);
10    } else {
11      console.log("Nota no vàlida");
12    }
13  }
```



```
14
15   get mitjana() {
16     const total = this.notes.reduce((acc, n) => acc + n, 0);
17     return this.notes.length ? (total / this.notes.length).
       toFixed(2) : "No hi ha notes";
18   }
19 }
20
21 class Estudiant {
22   #dni; // propietat privada
23
24   constructor(nom, dni) {
25     this.nom = nom;
26     this.#dni = dni;
27     this.assignatures = [];
28   }
29
30   get identificador() {
31     return `Estudiant: ${this.nom} (DNI: ${this.#dni})`;
32   }
33
34   afegirAssignatura(assignatura) {
35     this.assignatures.push(assignatura);
36   }
37
38   mostrarQualificacions() {
39     console.log(this.identificador);
40     this.assignatures.forEach(a => {
41       console.log(`- ${a.nom}: mitjana = ${a.mitjana}`);
42     });
43   }
44 }
45
46 // Ús
47 const anna = new Estudiant("Anna Martí", "12345678A");
48 const mates = new Assignatura("Matemàtiques");
49 const catala = new Assignatura("Català");
50
51 mates.afegirNota(7.5);
52 mates.afegirNota(9);
53 catala.afegirNota(8.2);
54
55 anna.afegirAssignatura(mates);
56 anna.afegirAssignatura(catala);
57
58 anna.mostrarQualificacions();
59   // Estudiant: Anna Martí (DNI: 12345678A)
60   // - Matemàtiques: mitjana = 8.25
61   // - Català: mitjana = 8.20
```

Aquest exemple mostra clarament l'**encapsulació de dades sensibles (DNI)**, l'**ús de get per obtenir la mitjana** de notes i com l'objecte Estudiant **agrega altres objectes** (Assignatura). A més, la funcionalitat `mostrarQualificacions()` actua com a **interfície pública**, amagant la complexitat interna.

3.3.2 Exemple 2: Jerarquia d'animals

Es vol programar un sistema per gestionar diferents animals d'un petit zoològic. Cada animal té un nom i és capaç d'emetre un so, però **el so canvia segons l'espècie**.

Es demana:

- Crear una **classe base Animal** amb un mètode `ferSo()` genèric.
- Crear subclasses com `Gos`, `Gat` i `Ocell`, que **hereten d'Animal** i **sobreescriuen el mètode `ferSo()`** per adaptar-lo a l'animal.
- Instanciar diversos animals amb noms propis.
- Emmagatzemar-los en un array.
- Recórrer l'array i cridar el mètode `ferSo()` de cadascun.

Aquest exercici ha de servir per entendre com funciona l'**herència**, el **polimorfisme** i l'ús de **`super()`** i **`this`** dins de la jerarquia d'objectes.

```
1 class Animal {
2   constructor(nom) {
3     this.nom = nom;
4   }
5
6   ferSo() {
7     console.log(`${this.nom} fa un so indefinit.`);
8   }
9 }
10
11 class Gos extends Animal {
12   ferSo() {
13     console.log(`${this.nom} borda: guau!`);
14   }
15 }
16
17 class Gat extends Animal {
18   ferSo() {
19     console.log(`${this.nom} miola: mèu!`);
20   }
21 }
22
23 class Ocell extends Animal {
24   ferSo() {
25     console.log(`${this.nom} canta: piu piu!`);
26   }
27 }
28
29 // Ús
30 const rex = new Gos("Rex");
31 const luna = new Gat("Luna");
32 const kiwi = new Ocell("Kiwi");
33
34 const animals = [rex, luna, kiwi];
```

```
35  
36 animals.forEach(a => a.ferSo());  
37 // Rex borda: guau!  
38 // Luna miola: mèu!  
39 // Kiwi canta: piu piu!
```

Aquest exemple mostra com diferents subclasses poden sobreescriure el mateix mètode (`ferSo`) per oferir comportaments diferents segons l'espècie, sense necessitat de modificar la classe base. És un cas clar de **polimorfisme en acció**.